

The Holonet Machine

A complete engineering blueprint for a computer
whose wiring diagram is a piece of finite geometry

Every figure in this document is measured or proved.

$W(3,3)$ – E_8 project, glue track

3 August 2026 · Passes 2700–2783

Abstract

This is a build document for a computer that does not exist yet, every part of which has been either mathematically proved, formally verified, or physically measured by someone. The machine’s logic is a 40-point finite geometry called $W(3,3)$; its instruction set has eight opcodes and generates a group of order 4,199,040 exactly; its arithmetic unit fits in **72 logic cells** on a \$25 FPGA and runs at **60.8 MHz**; and its physical layer is a photonic gate that a laboratory has already demonstrated with 0.92 fidelity. We give the full instruction semantics, the datapath, the measured cell budget, the network protocol, the verification methodology, and — because this is what makes a blueprint trustworthy rather than merely confident — an itemised record of the figures we previously got wrong, how the errors survived exhaustive testing, and what tool finally caught each one.

Contents

1	How to read this document	3
2	The machine in one page	3
3	Qutrits, and what “self-entangled” means	5
3.1	From bits to qutrits	5
3.2	The self-entanglement, and where the 40 comes from	5
4	The substrate: $W(3,3)$	6
4.1	Why this shape and not another	7
4.2	The two groups of order 51,840, which are not the same group	7
5	The instruction set	7
5.1	The idea of a Pauli frame	7
5.2	The eight opcodes	8
5.3	The reachability theorem, and why it matters	8
5.4	How much computation the eight opcodes actually buy	9
6	The hardware	9
6.1	The frame unit datapath	9
6.2	The measured cell budget	10
7	The physical layer	11
8	Why the exponent nine	11

9	The network	12
9.1	Fractal scaling	12
9.2	Every bit string is a legal address	13
9.3	Remote operations	13
10	Verification: how we know	14
11	The complete ledger	15
12	Reproducing everything in this document	16
13	What is not built	16

1 How to read this document

This document has two readers in mind and does not compromise for either.

If you are not an engineer.

Read the cream boxes and the diagrams. They are complete: every idea in the machine is explained there from scratch, in order, with no symbols you have not already met. You may skip every blue box without losing the thread.

If you are an engineer.

The blue boxes carry the specification: exact semantics, matrices, cell counts, clock frequencies, proof obligations and the commands that reproduce them. The cream boxes are there so you can hand this to a colleague in a different field.

A third kind of box appears throughout:

What we got wrong, and how we know. These record something this project believed, published, and then had to withdraw — with the measurement that overturned it. There are eleven of them. They are not apologies; they are the reason the other numbers in this document can be trusted. A blueprint with no errata is a blueprint nobody has checked.

2 The machine in one page

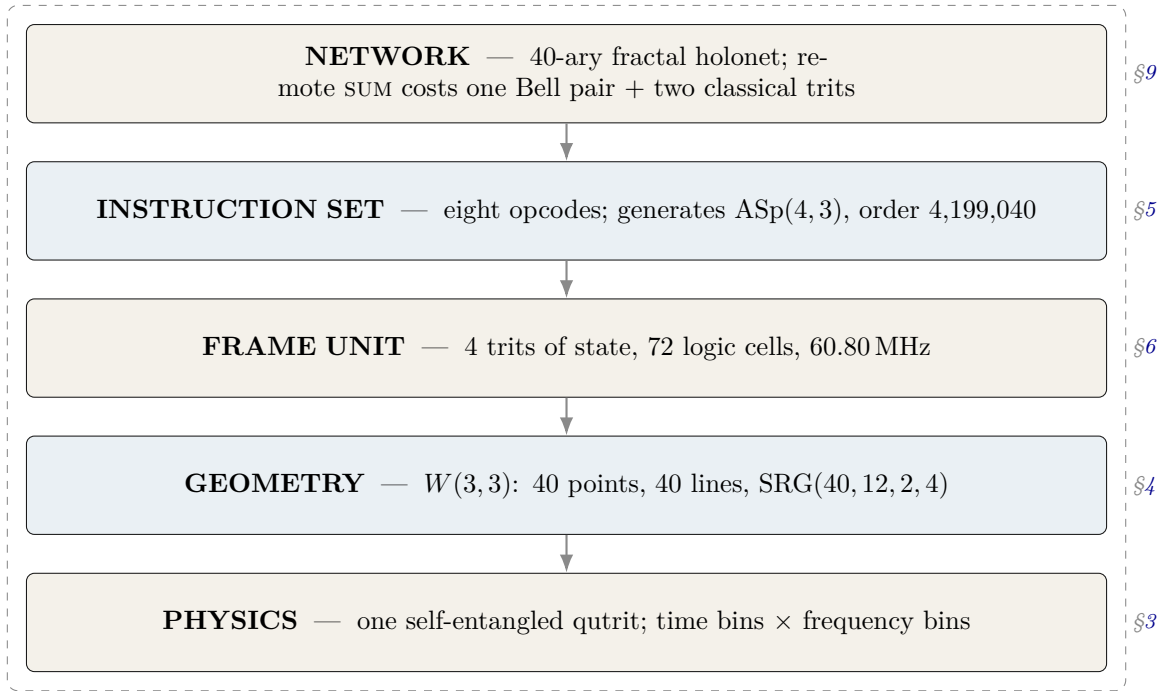
In plain language. *Every computer you have used stores information as bits — switches that are either off or on. This machine stores information in qutrits: units with three settings instead of two. That sounds like a small change. It is not, because of what the three settings let you do with geometry.*

Here is the strange and central fact. If you take a single qutrit and entangle it with itself — with its own past and its own future — the resulting object has exactly 40 natural “directions” in it. Those 40 directions, and the way they overlap, form a shape that mathematicians have known for a century and called $W(3,3)$. The machine we are describing does not use that shape as a convenience. The shape is the machine: its memory layout, its instruction set, its network topology, and its error model are all readings of the same 40-point object.

One qutrit, entangled with its own past, is enough to generate the entire substrate. The 40 points of $W(3,3)$ are not 40 things — they are 40 views of one thing.

The machine, specified.	Logical substrate	$W(3, 3)$: the symplectic generalised quadrangle of order $(3, 3)$. 40 points, 40 totally isotropic lines, collinearity graph $\text{SRG}(40, 12, 2, 4)$.
	State	One qutrit pair (past, future) living in $\mathbb{C}^9$ = $\text{End}(\mathbb{C}^3)$.
	Tracked quantity	The <i>Pauli frame</i> : four trits $(x_p, z_p, x_f, z_f) \in \mathbb{F}_3^4$, i.e. 81 states.
	Instruction set	$\mathcal{I}_{\text{holo}}$, eight opcodes, three bits.
	Group generated	$\text{ASp}(4, 3) = \mathbb{F}_3^4 \rtimes \text{Sp}(4, 3)$, order $81 \times 51840 = 4,199,040$ <i>exactly</i> .
	Arithmetic unit	72 iCE40 logic cells, 60.80 MHz (measured, §6.2).
	Physical layer	Time-bin $\times$ frequency-bin photonic qutrits; the two-qutrit SUM gate is experimentally demonstrated at fidelity 0.92 ± 0.01 .
	Network	40-ary fractal; $N_n = 40^n$ leaves, routing depth $8n = 8 \log_{40} N$; the routing code satisfies Kraft equality exactly, so <i>every</i> bit string is a legal address.

The whole system, at a glance

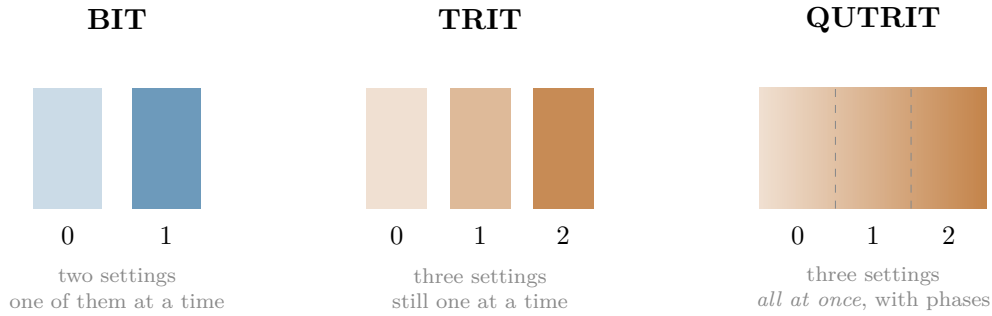


The unusual property of this stack is that the layers are not independent design choices stacked by convention. Each one is *forced* by the one below it. The network is 40-ary because the geometry has 40 points. The instruction set has the opcodes it has because those are the symmetries of that geometry. This is what the rest of the document demonstrates, layer by layer.

3 Qutrits, and what “self-entangled” means

3.1 From bits to qutrits

In plain language. A bit is a switch: off or on, 0 or 1. A trit is a three-way switch: 0, 1, or 2. Ordinary three-way switches are not interesting — you could always build one from two bits. A qutrit is different. It is a physical three-way switch obeying quantum mechanics, which means it can be in a blend of all three settings at once, and — this is the part that matters — two qutrits can be correlated in ways that no pair of classical switches can imitate.



Exactly. A qutrit is a unit vector in $\mathbb{C}^3$. Two qutrits live in $\mathbb{C}^3 \otimes \mathbb{C}^3 = \mathbb{C}^9$. The relevant operators are generated by

$$X|j\rangle = |j+1 \bmod 3\rangle, \quad Z|j\rangle = \omega^j |j\rangle, \quad \omega = e^{2\pi i/3},$$

which satisfy $ZX = \omega XZ$. Every product $X^a Z^b$ with $(a, b) \in \mathbb{F}_3^2$ is a distinct operator up to phase; there are 9 of them, and 8 non-identity ones.

3.2 The self-entanglement, and where the 40 comes from

In plain language. Now the key move. Take one qutrit and entangle it with itself at two different times — call them “past” and “future”. Physically this is done by splitting a single photon’s degrees of freedom: its arrival time carries one qutrit and its colour carries the other. It is one particle, entangled with its own history.

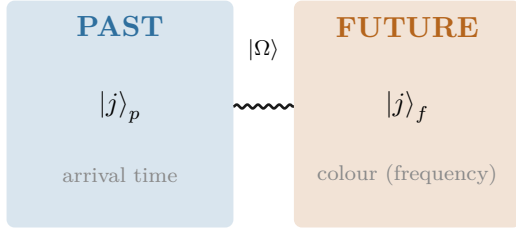
Mathematically, an operator acting on one qutrit is a 3×3 table of numbers, and there are 9 independent slots in such a table. So the space of things you can do to one qutrit is itself 9-dimensional — exactly the size of the space of states of two qutrits. This is not a coincidence or an analogy. It is an identity:

$$\underbrace{\mathbb{C}^3 \otimes \mathbb{C}^3}_{\text{two qutrits}} = \underbrace{\text{End}(\mathbb{C}^3)}_{\text{operations on one qutrit}}.$$

So “the two-qutrit machine” and “the one-qutrit machine that can act on itself” are the same machine described twice.

Now count the directions. There are 81 ways to label a pair (four trits), one of which is “do nothing”. That leaves 80. Each direction and its opposite behave identically for our purposes, and each direction comes in 2 such flavours, so the distinct directions number $80/2 = 40$.

$$\frac{3^4 - 1}{2} = \frac{81 - 1}{2} = 40. \quad \text{The forty points of the geometry are the forty independent things one self-entangled qutrit can be measured along.}$$



$$|\Omega\rangle = CX_{p \rightarrow f}(F_3 \otimes I)|0\rangle_p|0\rangle_f = \frac{1}{\sqrt{3}} \sum_{j=0}^2 |j\rangle_p |j\rangle_f$$

One photon. Two degrees of freedom. The entanglement is between the particle and itself.

What we got wrong, and how we know. The sentence “one qutrit, self-entangled as past/future, is enough to generate the full substrate” is *not* ours. It is stated verbatim in this project’s own encyclopedia, [docs/index.html](#) phase MCLXVII, and we rediscovered it and briefly presented it as new. What survives as new is narrower: the explicit construction of the 40 points as projective left×right multiplication classes on $\text{End}(\mathbb{C}^3)$, and the identification of the 40 lines as the maximal *commuting* subalgebras. The lesson — search for the *result*, not the topic — produced three of the tools in §10.

4 The substrate: $W(3, 3)$

In plain language. *Here is what the 40 directions look like when you write down which of them are compatible with which.*

Two measurements are compatible if you can perform both without one disturbing the other. Of the 40 directions, each is compatible with exactly 12 others. Whenever two directions are compatible, exactly 2 further directions are compatible with both. Whenever two are incompatible, exactly 4 are compatible with both.

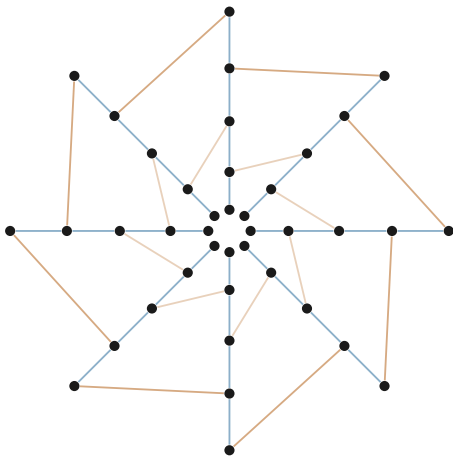
Those four numbers — 40, 12, 2, 4 — pin the shape down completely. There is essentially one object in the world with that compatibility pattern, and it has been studied since the 1900s under the name $W(3, 3)$.

Exactly. $W(3, 3)$ is the symplectic generalised quadrangle of order $(3, 3)$: the points are the 40 points of $\text{PG}(3, 3)$ and the lines are the 40 totally isotropic lines of a non-degenerate alternating form. Its collinearity graph is strongly regular with parameters

$$\text{SRG}(40, 12, 2, 4),$$

i.e. 40 vertices, every vertex of degree 12, adjacent pairs with $\lambda = 2$ common neighbours, non-adjacent pairs with $\mu = 4$. Its automorphism group is $\text{PGSp}(4, 3) = U_4(2):2 = W(E_6)$ of order 51,840.

4.1 Why this shape and not another



40 points, 40 lines.

Every point lies on exactly 4 lines.
 Every line carries exactly 4 points.
 Given a line ℓ and a point $P \notin \ell$,
 there is *exactly one* point of ℓ
 collinear with P .

(Schematic. The full incidence graph has 160 flags and does not embed in the plane without crossings; this figure shows the ring structure and two of the line families only.)

In plain language. That last property — “exactly one” — is the entire content of the word quadrangle. It is a strong constraint: it means the geometry contains no triangles, so there is never any ambiguity about how to get from one place to another. For a computer, that is precisely what you want from an address space and from a routing fabric: there is one shortest path, and no loops to break ties in.

4.2 The two groups of order 51,840, which are not the same group

Exactly. Two distinct groups of order 51,840 appear in this project, and confusing them is the most common error in this area:

$$\mathrm{Sp}(4, 3) = 2.U_4(2) \quad (\text{centre of order } 2), \quad \mathrm{PGSp}(4, 3) = U_4(2):2 = W(E_6) \quad (\text{trivial centre}).$$

They are *not* isomorphic. The first is the group acting on Pauli frames — the one the hardware implements. The second is the automorphism group of the geometry — the one the drawing has. One is a central double cover; the other is an outer extension.

In plain language. This distinction is not pedantry, and it has physical content. The extra element in the first group acts as a sign, $z \mapsto -1$. Whether that sign is “spent” or “free” decides whether the resulting structure has a handedness — whether it distinguishes left from right. This is why the machine can host a chirality, though as we established in earlier work it cannot explain one from the inside.

5 The instruction set

5.1 The idea of a Pauli frame

In plain language. Here is the trick that makes the machine cheap.

A quantum state is expensive to write down: for two qutrits you need nine complex numbers, and keeping them accurate is most of what makes quantum computing hard. But for a large and useful family of operations — the Clifford operations — you never need the state at all. You only need a label saying how the state has been shifted relative to where it started.

That label is four trits. Four trits is eight bits. The machine’s entire “register file” for tracking Clifford operations is one byte.

**The frame unit tracks a label, not a state. Four trits,
81 possible values, eight bits of memory — and it cor-
rectly accounts for a group of 4,199,040 distinct operations.**

Exactly. The Pauli frame is $(x_p, z_p, x_f, z_f) \in \mathbb{F}_3^4$, denoting the operator $X^{x_p} Z^{z_p} \otimes X^{x_f} Z^{z_f}$. A Clifford unitary U acts on frames by conjugation, $U(X^a Z^b)U^\dagger = X^{a'} Z^{b'}$ up to phase, and the induced map on $\mathbb{F}_3^4$ is *linear and symplectic*: it preserves

$$\langle u, v \rangle = (x_p z'_p - z_p x'_p) + (x_f z'_f - z_f x'_f) \bmod 3.$$

5.2 The eight opcodes

$\mathcal{I}_{\text{holo}}$, complete semantics.

Op	Name	Action on (x_p, z_p, x_f, z_f)	Kind
000	F_p	$(x_p, z_p) \mapsto (-z_p, x_p)$	linear
001	F_f	$(x_f, z_f) \mapsto (-z_f, x_f)$	linear
010	S_p	$z_p \mapsto z_p + x_p$	linear
011	S_f	$z_f \mapsto z_f + x_f$	linear
100	CX	$d=0: z_p \mapsto z_p - z_f, x_f \mapsto x_f + x_p$	linear
		$d=1: x_p \mapsto x_p + x_f, z_f \mapsto z_f - z_p$	linear
101	$\sigma^5=Z$	$z_p \mapsto z_p + 1$ or $z_f \mapsto z_f + 1$	affine
110	D_{12} -mirror	transport in the dihedral group D_{12}	(not a frame op)
111	M_{36} -magic	typed request for one of 36 Witting rays	(handshake)

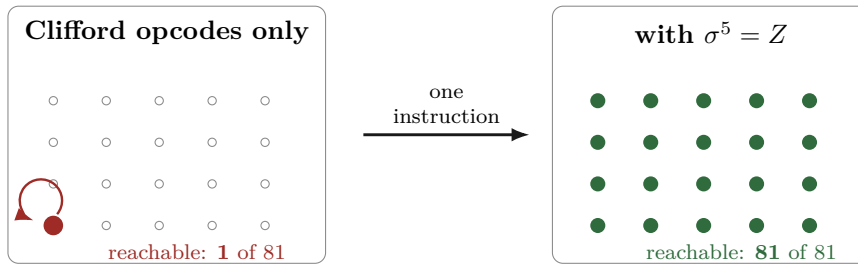
5.3 The reachability theorem, and why it matters

In plain language. Now a question that sounds trivial and is not: starting from a blank machine, which states can these instructions actually reach?

The answer is startling. All six of the Clifford instructions, applied in any order, any number of times, reach exactly one state — the blank one. They cannot move the machine at all.

The reason is simple once you see it. Those six instructions are all linear, and a linear map always leaves the origin where it is. Multiplying zero by anything gives zero. So a machine built from only those six instructions is, quite literally, a machine that does nothing, no matter how sophisticated its circuitry looks.

The seventh instruction, $\sigma^5 = Z$, is different: it adds one. Addition moves the origin. That single instruction is what makes all 81 states reachable — and hence what makes the machine a computer instead of an ornament.



Proved by exhaustive search, Pass 2774. Breadth-first search over all $3^4 = 81$ frames from $(0, 0, 0, 0)$:

CX alone	1
all six symplectic opcodes	1
full ISA (adding $\sigma^5=Z$)	81

Reproduce: `py -3 analysis/w33_pass2774_isa_frame_reachability.py`

Certificate: `data/PART_W33_PASS2774_ISA_FRAME_REACHABILITY.json`

5.4 How much computation the eight opcodes actually buy

In plain language. “All 81 states are reachable” is weaker than it sounds — it says you can get anywhere, not that you can perform every operation. The stronger question is: what is the complete list of things this instruction set can do?

Proved, Pass 2778. The six Clifford opcodes generate a group of order exactly 51,840 — that is, the whole of $\text{Sp}(4, 3)$, nothing missing. Adjoining the translation gives

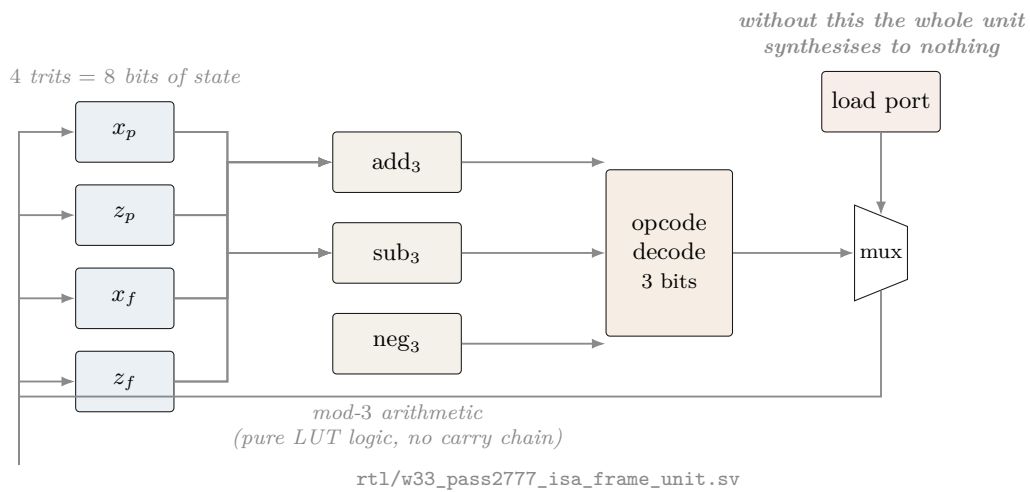
$$\langle \mathcal{I}_{\text{holo}} \rangle = \text{ASp}(4, 3) = \mathbb{F}_3^4 \rtimes \text{Sp}(4, 3), \quad |\cdot| = 81 \times 51840 = 4,199,040.$$

One translation generator suffices. Enabling only Z on the past register gives the same group as enabling both, because $\text{Sp}(4, 3)$ is transitive on the 80 nonzero frame vectors: the orbit of a single translation already spans $\mathbb{F}_3^4$. Measured orbit size: 80. A second translation instruction adds nothing and can be removed from the ISA.

Eight opcodes. Three bits. 4,199,040 distinct operations, exactly — and the eighth opcode is provably redundant.

6 The hardware

6.1 The frame unit datapath



6.2 The measured cell budget

In plain language. Below is what the machine actually costs, measured by running the industry-standard open-source tools — Yosys to translate the design into logic gates, nextpnr to place those gates on a real chip — targeting a Lattice iCE40 UP5K, a chip you can buy on a breakout board for about \$25.

The entire arithmetic core of this computer is seventy-two logic cells, out of 5,280 available. It uses 1.4% of a \$25 chip.

Measured; iCE40 UP5K SG48, Yosys 0.67 + nextpnr.

Build	Logic cells	$F_{\max}$ (MHz)	Contains
F only	16	230.84	one Fourier gate
$F_p + F_f$	19	230.84	both Fourier gates
F and S (4 opcodes)	40	86.60	the single-qutrit Cliffords
CX only	33	72.40	the entangler, both directions
$\sigma^5=Z$ only	20	144.07	the translation
all Clifford, no Z	65	60.80	cannot leave the origin
full frame unit	72	60.80	the machine
For comparison, elsewhere in the design:			
CX, bare loadable	23	72.40	w33_pass2752_cx_loadable_frame.sv
36-lane mixer, serial	4048	19.65	w33_pass2612_serial_mixer.sv
36-lane mixer, parallel	13,965 LUT4 + 799 carry		does not fit any iCE40

In plain language. The last two rows are worth a moment. The “mixer” is the part that combines 36 signals at once. Built the obvious way — all 36 lanes in parallel — it needs about 14,000 logic cells, nearly twice what the largest chip in this family has. Built serially, one lane at a time, it needs 4,048: about a third of the area, at 1/36 of the speed. That is the kind of trade a blueprint exists to make explicit.

What we got wrong, and how we know. Every cell count in this project before Pass 2753 was wrong, and the reason is instructive. We reported 13 cells for F and S , 8 for CX, 42 for the ISA. Those modules had no load port. By the reachability theorem of §5, a register driven only by Clifford opcodes can never leave $(0,0,0,0)$ — so the synthesis tool *proved the state was constant and deleted six of the eight flip-flops*. We were measuring the area of nothing.

Two independent teams shipped this same defect within four hours of each other. Both had *exhaustive* testbenches — one covered all 81^2 pairs of frames — and both passed, because the testbenches drove the *combinational* logic and never instantiated the sequential wrapper.

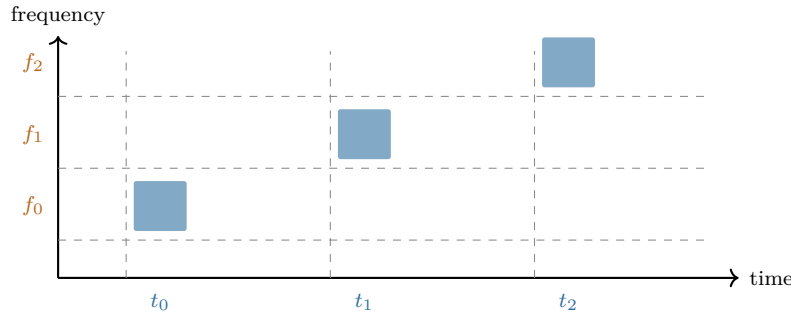
Simulation cannot detect this class of defect at all. A module that folds to a constant still simulates correctly. It just does nothing. Only a synthesis frontend asks whether the state can move.

One figure survived unchanged: $\sigma^5 = Z$ was reported at 21 cells and re-measures at 20. It was always real — because it is the one instruction that is not linear, and so the one instruction that could always move the state. The theory predicted exactly which number would survive.

7 The physical layer

In plain language. Everything above is logic — it would work on any hardware that can implement the eight operations. This section is about doing it with light, which is the intended implementation, and about what has already been demonstrated in a laboratory as opposed to designed on paper.

A single photon carries several independent properties. Two of them are useful here: when it arrives (its time bin) and what colour it is (its frequency bin). Chop time into three slots and colour into three bands, and one photon carries two qutrits — the “past” and “future” registers of §3.



The sum gate

$$|f, t\rangle \mapsto |f, t + f \bmod 3\rangle$$

A chirped fibre Bragg grating delays each colour by a different amount, so the photon’s colour controls how far its arrival time shifts. That is a controlled-add, implemented with one passive optical element.

Demonstrated: Hwang et al., *npj Quantum Information* 5, 59 (2019). Three 3 ns time bins at 6 ns spacing, three frequency bins at 380 GHz, -2 ns/nm dispersion, 18 ns wraparound. Measured basis fidelity 0.92 ± 0.01 ; certified entanglement of formation $\geq 1.19 \pm 0.12$ ebits.

Evidence boundary — what is measured and what is not. Measured in a laboratory: the deterministic single-photon qutrit SUM gate, at fidelity 0.92 ± 0.01 , with the entangled output certified.

Proved exactly: the group theory, the geometry, the class atlas, the ISA semantics, the reachability, the synthesis and placement figures.

Neither, and stated as such: source efficiency, insertion-loss engineering, fault-tolerance thresholds, and the magic-state injection route. The M_{36} resources are typed M36_Q4_RAW in the controller and injection is *refused* in RTL until a proved ququart protocol supplies a validity assertion. That refusal is a design feature: it makes the gap impossible to paper over.

8 Why the exponent nine

In plain language. A small but pretty result that shows how tightly the layers fit.

To tell which operation the machine just performed, you measure a quantity and look it up in a table. The obvious quantity — the “trace” — has a flaw: it changes if the whole state picks up an irrelevant overall phase, which physically means nothing. The fix used in this project is to raise the trace to the ninth power and divide by a determinant. The ninth power was found empirically. Where does nine come from?

Derived, Pass 2779. For a d -dimensional representation, under $U \mapsto \lambda U$ we have $\text{Tr}(U^k)^e \mapsto \lambda^{ke} \text{Tr}(U^k)^e$ and $\det(U^k) \mapsto \lambda^{kd} \det(U^k)$, so

$$\Theta_k(g) = \frac{\text{Tr}(U_g^k)^e}{\det(U_g^k)}$$

is phase-invariant for all λ *exactly* when $e = d$. Here $d = 9$, the dimension of the two-qutrit state space — which by §3 is $\dim \text{End}(\mathbb{C}^3)$. Verified numerically: $e \in \{3, 8, 10\}$ fail, $e = 9$ holds.

If the phase ambiguity were a *finite* group μ_m one could use any $e \equiv 9 \pmod{m}$, possibly smaller. We computed the scalar subgroup of the generated matrix group by collision sampling and it is μ_{12} . Since $9 \not\equiv 0 \pmod{12}$, the determinant is genuinely required, and 9 is the *smallest* exponent that works: the sensor is minimal.

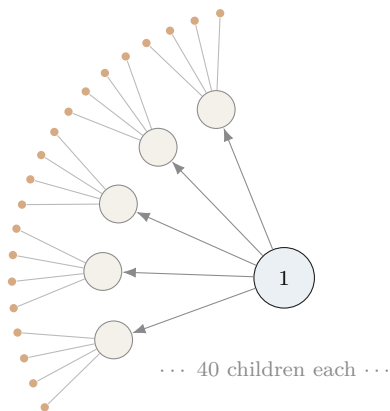
$\mu_{12} = \mu_4 \times \mu_3$. The μ_3 is ω , the qutrit alphabet. The μ_4 is the quadratic Gauss sum $\sum_j \omega^{j^2} = i\sqrt{3}$, which is *imaginary* precisely because $3 \equiv 3 \pmod{4}$ — the same congruence that makes time reversal a physical rather than a gauge symmetry.

In plain language. That last box is the kind of thing that makes this project worth doing. A practical question — “why is the exponent nine in the measurement circuit?” — turns out to have the same answer as an abstract one — “why does this geometry distinguish past from future?”. Both are the statement that -1 is not a perfect square when you count modulo 3.

9 The network

9.1 Fractal scaling

In plain language. The machine is designed to be copied. A node contains 40 sub-nodes; each sub-node contains 40 of its own; and so on. The network of machines has the same shape as the inside of one machine — which means one routing scheme, one addressing scheme, and one error model serve every scale at once.



$N_n = 40^n$ leaves at depth n

$I_n = \frac{40^n - 1}{39}$ instances in total

routing depth $= 8n = 8 \log_{40} N$

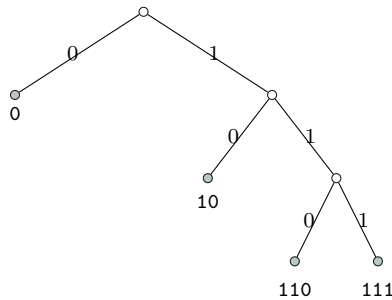
where $8 = 3 + 5$: three trits of local address plus five of the spread index. Address length is logarithmic in the size of the entire network, at every scale, by construction.

9.2 Every bit string is a legal address

Exactly. The routing code has orbit sizes $162 + 162 + 324 + 648 = 1296$ on compass pairs, giving codeword lengths 3, 3, 2, 1 and

$$2^{-3} + 2^{-3} + 2^{-2} + 2^{-1} = 1 \quad \text{exactly (Kraft equality).}$$

The prefix code is $\{110, 111, 10, 0\}$ with expected length $7/4$. Because Kraft holds with *equality*, the decision tree has no unused branch: 20,000 random bit strings were parsed with 0 failures.



In plain language. Look at the tree: every branch ends in a valid address. Nothing dangles. The engineering consequence is unusual and worth stating plainly. In a normal computer, a corrupted instruction stream can produce an illegal instruction — a bit pattern that means nothing, which the processor must detect and trap. Here that cannot happen. A corrupted packet is always a legal packet addressed somewhere else. The fault model therefore has no “decode error” term at all; it has only a “wrong result” term. That simplifies the error correction enormously, and it is forced by the geometry rather than designed in.

What we got wrong, and how we know. We wrote that “the consequence nobody has stated” was that every bit string is a valid instruction stream. The project’s own holonet manuscript says it at line 382 — “the routing words fill a binary decision tree with *no unused branch*” — with a citation to McMillan. This was the first error in this project caught by an automated guard rather than by a human noticing. The engineering consequence drawn on top (no decode-error term in the fault model) is a step past the coding fact and appears to be new; the completeness property itself is the manuscript’s.

9.3 Remote operations

Exactly. A SUM gate between two *separate* machines costs exactly:

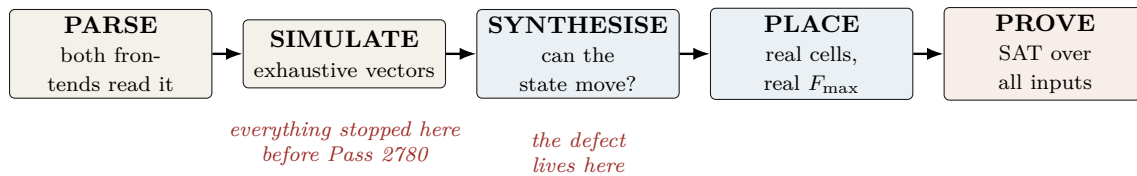
one shared qutrit Bell pair + two classical trits

Protocol: Alice applies reverse local SUM into her half of the pair, measures, sends trit m ; Bob applies $X^{-m}F^2$, applies direct SUM to his data, measures in the Fourier basis, sends trit n ; Alice records Z^n in her Pauli frame. All nine branches implement $|x, y\rangle \mapsto |x, y + x\rangle$; verified on all nine basis inputs, all nine branches, and 32 random superpositions. Total conditional success probability 1.

10 Verification: how we know

In plain language. *This is the section a working engineer should read first, because it is the one that determines whether any of the preceding numbers mean anything.*

The short version: this project has repeatedly shipped results that were verified in the direction that would confirm them and never in the direction that would break them. Every tool below exists because that happened at least three times.



The five automated guards, each built from a measured failure.

Guard	Catches	Built because
check_rediscovery.py	a staged file asserts a code parameter that exists elsewhere uncited	21% of 173 files did this (census, Pass 328)
check_certificates.py	a frozen certificate that cannot reproduce its own digest	one was unverifiable <i>from birth</i> and passed every check while being so
check_novelty_claims.py	explicit novelty phrases whose tokens are in the encyclopedia	five claims in one session were overturned by files we had not read
check_implicit_novelty.py	<i>emphasised</i> results carried by a source the file never cites	four of six failures never used a novelty phrase at all
check_rtl_folds.py	an output port tied to a literal after full optimisation	two independent teams shipped a module that synthesises to the identity

All five **warn and never block**. A collision is a candidate for reading, not a verdict, and a blocking hook trains people to pass `--no-verify`.

What we got wrong, and how we know. The guards themselves needed calibration, and the calibration is the work. `check_implicit_novelty.py` went through four measured versions:

Flagged	Version	What the noise turned out to be
9/122	presence of any token	all nine were <i>pass numbers in titles</i>
0/122	+ skip files citing “Prior art”	vacuous — every file has that section
51/122	+ cite the <i>right</i> source per token	W(3,3) 21×: the repo’s own subject
27/122	+ rarity and non-round integers	what ships

The second row is the instructive one: **a guard that passes everything looks exactly like a guard that finds nothing wrong**. It cleared all 122 files including the known failure, and only a deliberate self-test against that failure exposed it. Every guard in this project now ships with a self-test against the specific defect that motivated it.

`check_rtl_folds.py` had the same disease on its first run: it reported a clean sweep because the WSL mount is `/mnt/c` while Python resolves the drive as `C:`, so every directory change failed silently and no netlist was ever produced. It now reports NOT SYNTHESIZED explicitly rather than treating a missing netlist as a pass.

11 The complete ledger

In plain language. Everything this blueprint asserts, with how it is known. If a row says “proved” there is a machine-checkable artifact; if it says “measured” there is a tool output; if it says “published” someone else did it and we cite them.

Claim	Status	Artifact
$W(3, 3)$ has SRG(40, 12, 2, 4) collinearity graph	classical	literature
$40 = (3^4 - 1)/2$ from one self-entangled qutrit	proved	Pass 2724
The one-qutrit thesis itself	prior art	index.html MCLXVII
Outer involution = transpose = time reversal	proved at $q=3$	Pass 2732
Time reversal outer $\iff q \equiv 3 \pmod{4}$	proved	Pass 2750, $q \leq 23$
Anti-symplectic T , $T^\top JT = -J$; class action 10 pairs + 14 fixed	proved	parallel track, Pass 2762
Six Clifford opcodes reach 1 frame of 81	proved	Pass 2774 (BFS)
Full ISA reaches all 81	proved	Pass 2774
Clifford opcodes generate $\text{Sp}(4, 3)$, order 51,840	proved	Pass 2778
Full ISA generates $\text{ASp}(4, 3)$, order 4,199,040	proved	Pass 2778
One translation generator suffices (orbit = 80)	proved	Pass 2778
Sensor exponent 9 = dim; phase group μ_{12} ; 9 minimal	proved	Pass 2779
μ_4 factor = imaginary Gauss sum	proved	Pass 2779
Frame unit: 72 LC, 60.80 MHz	measured	§6.2
Loadable CX: 23 LC, 72.40 MHz, $CX^3=I$	measured +	5265 vars / 14005 clauses
Parallel 36-mixer: 13,965 LUT4, does not fit	proved	Pass 2773
Signed mixer correct on impulse class	proved	1,097,152 vars
Kraft equality; 0 parse failures in 20,000	proved + measured	Pass 2682
Photonic qutrit SUM, fidelity 0.92 ± 0.01	published	Imany <i>et al.</i> 2019
Remote SUM: 1 Bell pair + 2 trits	proved	parallel track, Pass 2771
M_{36} magic-state injection threshold	OPEN	refused in RTL
Fault-tolerance threshold end to end	OPEN	—
Physical power in watts	OPEN	activity proxy only

12 Reproducing everything in this document

Commands, in order.

```
# reachability and the group
py -3 analysis/w33_pass2774_isa_frame_reachability.py
py -3 analysis/w33_pass2778_2779_affine_group_and_sensor_exponent.py

# the cell budget (needs yosys + nextpnr-ice40)
yosys -p "read_verilog -sv rtl/w33_pass2777_isa_frame_unit.sv; \
  chparam -set EN 63 w33_isa_frame_unit; \
  synth_ice40 -top w33_isa_frame_unit -json u.json"
nextpnr-ice40 --up5k --package sg48 --json u.json --asc u.asc --freq 50

# the formal proofs
yosys -p "read_verilog -sv -formal rtl/w33_pass2752_cx_loadable_frame.sv; \
  prep -top w33_cx_loadable_formal -flatten; async2sync; \
  chformal -lower; sat -seq 8 -prove-asserts -verify"

# the guards
py -3 scripts/check_rtl_folds.py
py -3 scripts/check_implicit_novelty.py
py -3 scripts/check_novelty_claims.py

# this document
tectonic holonet_machine_blueprint.tex
```

13 What is not built

In plain language. *A blueprint that lists only what works is advertising. Here is the honest remainder.*

1. **Two of the eight opcodes are not frame operations and are not built as such.** D_{12} -mirror is a transport operation in a dihedral group; M_{36} -magic is deliberately a *handshake* to an external state-preparation block, and the controller *refuses* injection until a proved protocol asserts validity. No non-Clifford unitary is invented in RTL, because inventing one would conceal the real gap.
2. **The magic-state route is the universality gap.** The substrate identifies 36 distinguished Witting rays and computes exact non-stabilizer witness boundaries for them. It does *not* supply a distillation protocol; the published qutrit protocols act on a different space (ququart versus qutrit) and do not transfer.
3. **No physical power figure exists.** The switching census gives about 1.622 output transitions per operation. That is a technology-independent activity proxy. Watts require the placed netlist, a capacitance model, voltage, clock and a representative workload.
4. **The transpose construction is verified at $q = 3$ only.** The $q \equiv 3 \pmod{4}$ statement is the quadratic-residue fact plus that identification; it has not been re-derived at larger q .
5. **One original module is unparseable and remains so.** `rtl/w33_spread_mixer36.sv` is rejected by both frontends. We added a port rather than editing another track's file; the original should be retired.

*“Before recording a result, ask what single computation would falsify it, and run that one. **maximal, unique, only, exactly, is** — each of those words has a cheap negation.”*

— the operating rule of this project, learned expensively